GOLD: More Arrays (Wed Sep 16)

Arrays and Methods

Generalizations: arrays and methods

You already saw `int[] computeHistogram(...)` and `int computeSum(int[] a)`.

The full details:

- A method can **take an array as a parameter**:
  - `int computeSum(int[] a)`,
  - `boolean allPositive(double[] xs)`
- A method can **return an array**:
  - `int[] computeHistogram(String[] words)`,
  - `int[] copyDoubled(int[] a)`
- An array of type `T[]` always has elements of type `T`
  - `int[]` contains `int`,
  - `String[]` contains `String`.

```
int[] copyDoubled(int[] a) {
    int[] cpy = new int[a.length];
    for (int i=0; i < a.length; i++)
        cpy[i] = 2 * a[i];
    return cpy;
}

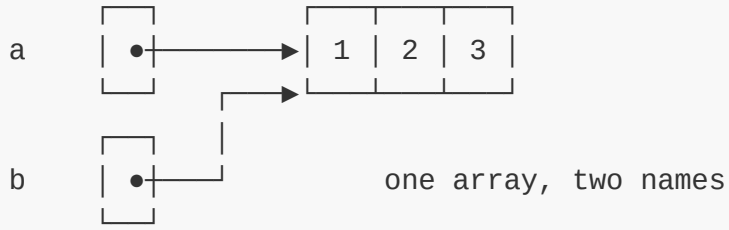
void main() {
    int[] arr = {1, 2, 3, 4};
    int[] doubled = copyDoubled(arr);
    IO.println(Arrays.toString(doubled));
    return;
}
```

Aliasing and Copying

Two names, one array

An array variable holds an **arrow** (a reference), not the data. So what does `=` copy?

```
int[] a = {1, 2, 3};
int[] b = a;          // ?
b[0] = 99;
IO.println(a[0]);     // ?
```



`b = a` copies the **arrow**.

- There is still exactly one array;
- `a` and `b` are two names for it, which we call **aliases**.
- Change the array through either name and both variables refer to the modified array.

Pause and predict

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
b[0] = 99;
IO.println(a[0]);
```

Q. What prints?

- **A.** `1`
- **B.** `99`
- **C.** Compile error
- **D.** Crash

Three consequences

- **`==` compares arrows.**
  - `a == b` is `true` (same array);
  - `a == c` is `false` for a different array with the same contents.
  - To compare contents: `Arrays.equals(a, c)` or write your own loop

- **A real copy needs new boxes.**
  - To make a copy: `int[] d = a.clone();`
  - or write a loop that copies element by element.
  - Afterwards, assigning `d[1] = 42` does not change `a[1]` .

- **Methods receive the arrow.** Pass an array to a method and the method can change your array
  - There is no copy when passing or return arrays.
  - That's how `computeHistogram` could read your array of words
  - One rule: **Java always copies the value.**
    - For primitives (`int`, `double`) this is the value itself;
    - For arrays and Strings the value is the arrow.

Example:

```
void zeroOut(int[] xs) {
    for (int i = 0; i < xs.length; i++) {
        xs[i] = 0;
    }
}

void main() {
    int[] a = {1, 2, 3};
    int[] b = a;                        // copies the ARROW, not the boxes
    b[0] = 99;
    IO.println(a[0]);                   // 99: a and b are one array with two names

    IO.println(a == b);                 // true:  same array
    int[] c = {99, 2, 3};
    IO.println(a == c);                 // false: different arrays, same contents
    IO.println(Arrays.equals(a, c));    // true:  compares the contents

    int[] d = a.clone();                // a real copy: new boxes, same values
    d[1] = 42;
    IO.println(a[1]);                   // 2: a is untouched

    zeroOut(a);
    IO.println(Arrays.toString(a));     // [0, 0, 0] the method changed our array
}
```

Remember: `=` never copies an array. If you need your own, call `clone()` or write a loop.

2-D Arrays

Arrays of arrays

An array element can be **any** type, including another array.

- This gives a **grid** (a 2-D array):

```
int[][] matrix = new int[3][7];     // 3 rows, each an int[7]
```

Read it as: `matrix` holds **3** references, and each one points to an `int[7]`.

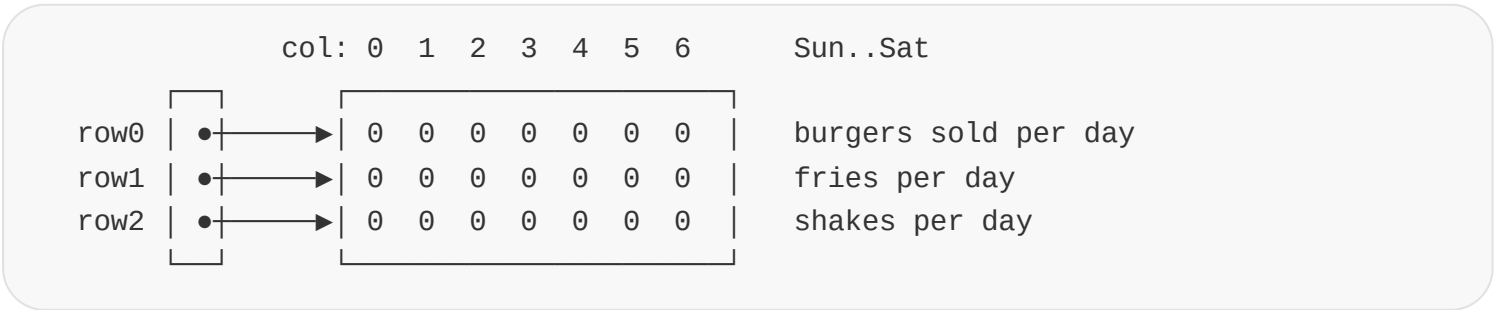
- You select an element with **two** indices. Row first, then column:

```
matrix[1][4] = 99;                  // row 1, column 4
```

A picture of a 3×7 grid

`int[][] sales = new int[3][7]` has:

- three rows,
- each a 7-element `int[]`,
- every cell `0`:



- `sales.length` is **3** (the number of rows)
- `sales[0].length` is **7** (the columns in row 0)
- `sales[i][j]` is row `i`, column `j`

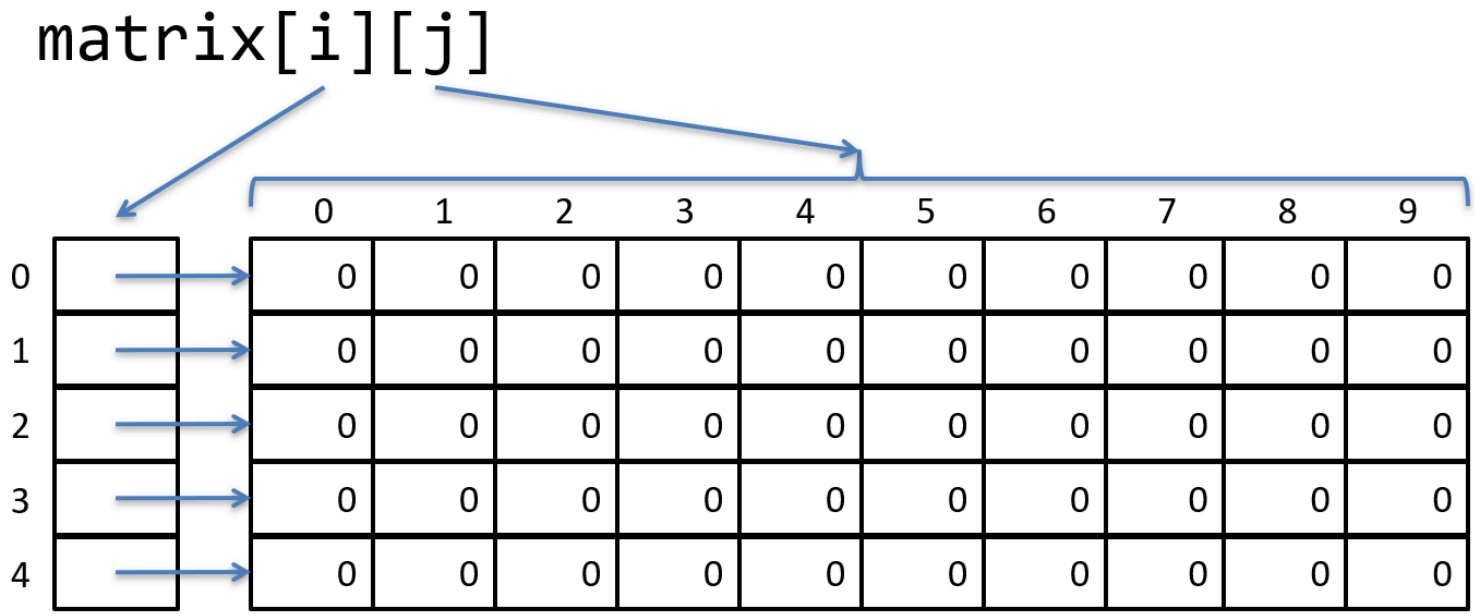
Static Initialization

You can use static initialization with these arrays too:

- Creates a 2D matrix with 5 rows and 10 columns

```
{ { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 } }
```

- Elements of `matrix[i]` are arrays of (10) ints.



Array Activity

Read the grid

```
int[][] sales = {
    {12, 15,  9, 14, 20, 31, 28},   // row 0: burgers
    { 8, 11,  7, 10, 16, 22, 19},   // row 1: fries
    { 5,  6,  4,  7,  9, 14, 12},   // row 2: shakes
};
```

With your neighbor:

1. How can you find the total number of elements in a 2D-array?

2. What is `sales[1][4]`?

3. Which `[row][col]` holds the value `31`?

Pause and predict (poll)

Q. What is `sales[1][4]`?

- **A.** `20`
- **B.** `16`
- **C.** `9`
- **D.** `14`

Nested Loop

Summing a grid: the nested loop

To sum (or process) every cell, loop rows on the outside and columns on the inside.

Then `matrix.length` is the outer bound and `matrix[i].length` as the inner bound:

```
int computeSum(int[][] matrix) {
    int sum = 0;
    for (int i = 0; i < matrix.length; i++)          // each row
        for (int j = 0; j < matrix[i].length; j++)   // each column of that row
            sum += matrix[i][j];
    return sum;
}
```

For the `sales` grid, this returns **279**

```
int computeSum(int[][] matrix) {
    int sum = 0;
    for (int i = 0; i < matrix.length; i++)          // each row
        for (int j = 0; j < matrix[i].length; j++)   // each column of that row
            sum += matrix[i][j];
    return sum;
}

void main() {
    int[][] sales = {
        {12, 15,  9, 14, 20, 31, 28},   // row 0: burgers
        { 8, 11,  7, 10, 16, 22, 19},   // row 1: fries
        { 5,  6,  4,  7,  9, 14, 12},   // row 2: shakes
    };
    IO.println(computeSum(sales));
}
```

Higher Dimensions

Higher dimensions:d a word of caution

Two dimensions generalize to as many as you like:

```
double[][][] cube = new double[10][10][10];                 // 1,000 cells
double[][][][] spaceTime = new double[100][100][100][100];
```

But watch the multiplication: that `spaceTime` array is 100^4 = **100,000,000** `double`s ≈ 800 MB.

Dimensions multiply *fast*.

Uses of 2D Arrays

- Representing:
  - A grid-based game, like tic-tac-toe or chess
  - A set of distances between pairs of cities
  - Matrices in linear algebra
  - Other tabular data

Ragged Arrays

Ragged arrays: rows of different lengths

Java does **not** require every row to be the same width. You can build each row separately:

```
int[][] matrix = new int[5][];     // 5 rows, none created yet (all null)
matrix[0] = new int[7];
matrix[1] = new int[10];
matrix[2] = new int[4];
matrix[3] = new int[6];
matrix[4] = new int[9];
```

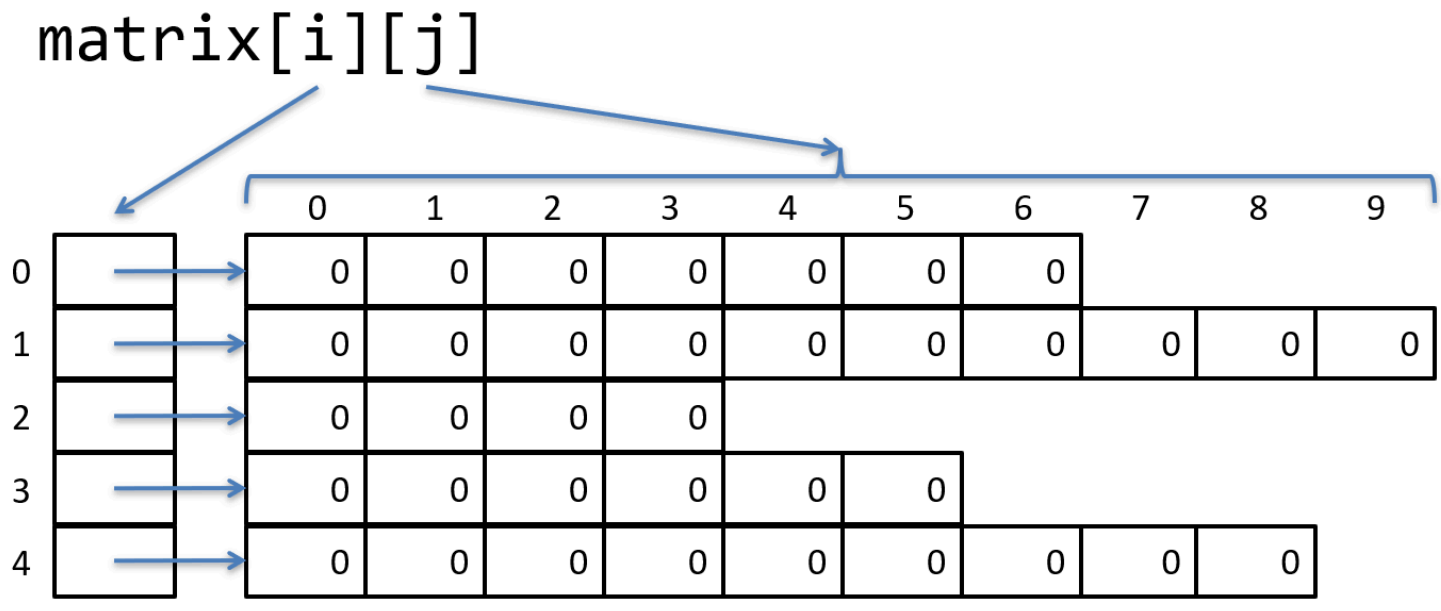
Note that the construction of the array is now `new int[5][]` with the **second brackets empty.**

You can't leave the first empty but specify the second.

Our picture now looks like

```
      ┌──┐   ┌──────────────────┐
row0  │ ●┼──►│ 0 0 0 0 0 0 0 └──┘          (7)
row1  │ ●┼──►│ 0 0 0 0 0 0 0 0 0 0 │       (10)
row2  │ ●┼──►│ 0 0 0 0 │──────────┘         (4)
row3  │ ●┼──►│ 0 0 0 0 0 0 └──┐            (6)
row4  │ ●┼──►│ 0 0 0 0 0 0 0 0 0 │        (9)
      └──┘   └──────────────────┘
```

or



This is a **ragged** (or *jagged*) array

- useful when rows genuinely have different sizes
- it saves storage versus padding to a rectangle.

What applications can you think of?

Pause and predict

```
int[][] m = new int[5][];
IO.println(m[0].length);
```

What happens?

- **A.** `0`
- **B.** `5`
- **C.** Crash: `NullPointerException`
- **D.** Compile error

For-each

The `for`-each loop

When you want to visit **every** element and don't need the index, there's a cleaner loop.

We held it back until now because it's built *for* arrays:

```
void main() {
    int[] scores = {88, 92, 75, 100, 64};

    int sum = 0;
    for (int value : scores)      // "for each value in scores"
        sum += value;

    IO.println(sum);
}
```

Read the `:` as "in."

It walks the whole array, binding each element to `value` in turn.

There is no index, no `length`, no off-by-one to worry about.

The drawback is that we cannot change the contents of the array.

`value` is a local variable containing a copy of each element, not a reference to anything.

This looks a lot like the Python version...

```
scores = [88, 92, 75, 100, 64]

sum = 0
for value in scores:       # "for each value in scores"
    sum += value

print(sum)
```

`for`-each on a grid

`for`-each nests, and because it reads each row as `int[]`, it handles ragged grids for free:

```
int computeSum(int[][] matrix) {
    int sum = 0;
    for (int[] row : matrix)      // each row is an int[]
        for (int elem : row)      // each element of that row
            sum += elem;
    return sum;
}
```

For-each Discussion

Predict: the `for`-each loop

```
int[] nums = {1, 2, 3};

for (int x : nums)
    x = x * 10;            // try to multiply each element by 10

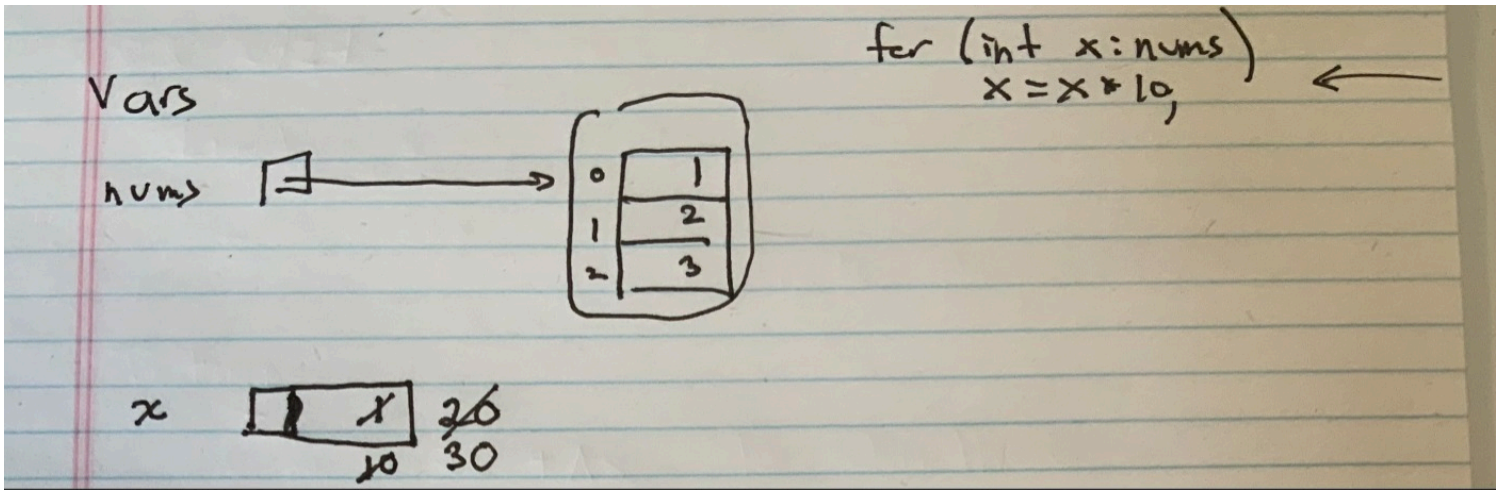
IO.println(Arrays.toString(nums));    // what prints?
```

Pause and predict (poll)

Q. What prints?

- **A.** `[10, 20, 30]`
- **B.** `[1, 2, 3]`
- **C.** Compile error
- **D.** `[10, 2, 3]`

Why doesn't modifying `x` have any effect on `nums`?



Each element of `nums` is copied into `x` so changing `x` by reassignment has no effect on the elements of `nums`.

With multi-dimensional arrays, the picture is a little more complex:

Consider the snippet:

```
void main() {
    int[][] grid = { {1, 0}, {0, 1} };

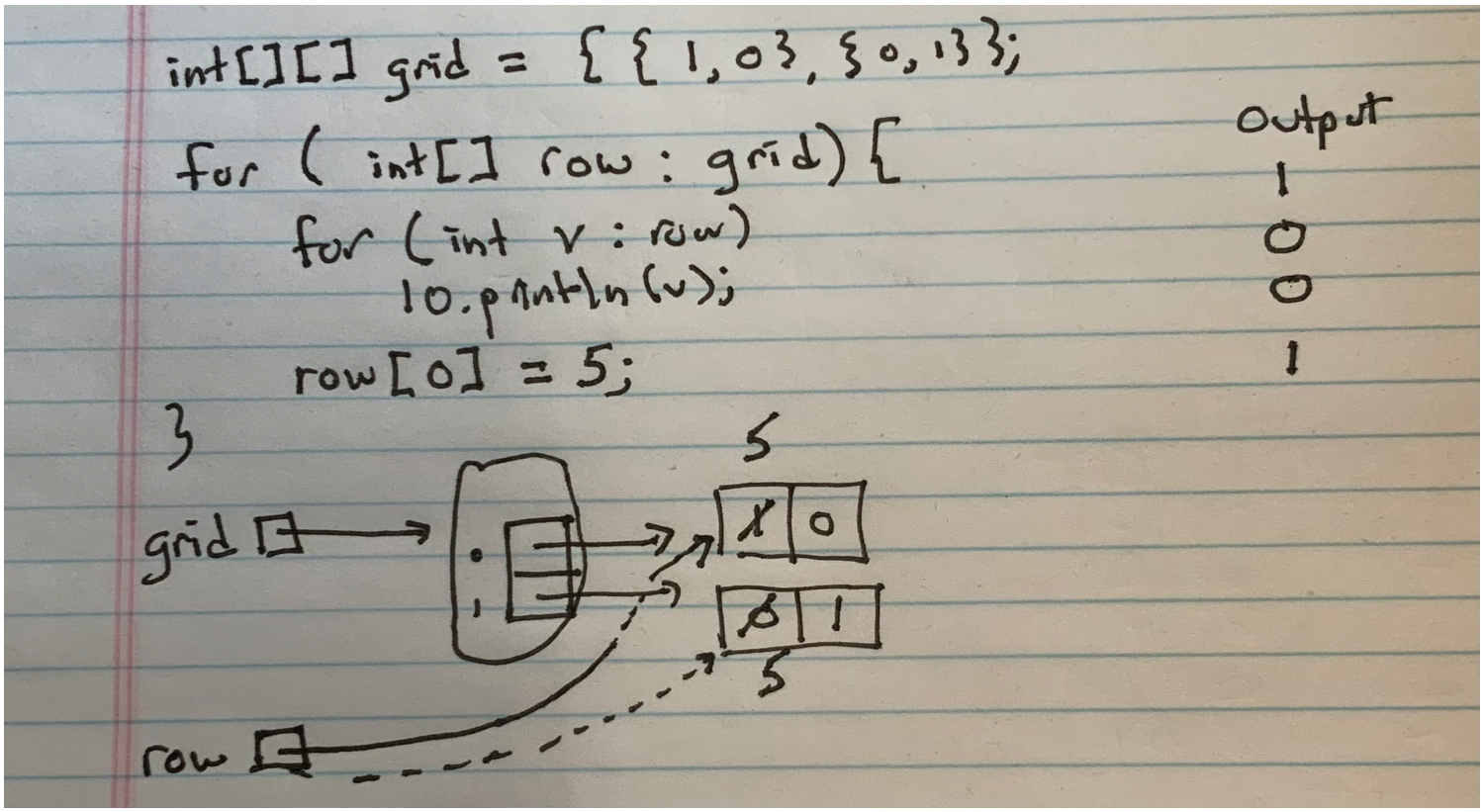
    for (int[] row : grid) {
    for (int v : row)
        IO.println(v);
    row[0] = 5;
    }

    for (int[] row : grid)
        IO.println(Arrays.toString(row));
}
```

The nested loops will print the contents of grid, one item per line:

```
1
0
0
1
```

But `row[0] = 5` can actually change the underlying grid. How?



- When we copy each element of `grid` into `row`, the value being copied is a **location** in memory.

- We cannot do anything to overwrite the elements of `grid` (entire rows) but we can manipulate whatever object that `row` is currently referencing.

- In particular, we can change the first element of each row to `5`.

This is a consequence of copying reference type values: multiple variables sharing the same underlying object.

ArrayList: First Look

A first look at generics

Before `ArrayList`, one new notation.

A **generic** class is *parameterized* by a type you fill in with angle brackets:

```
ArrayList<String>   names    // an ArrayList that holds Strings
ArrayList<Integer>  scores   // an ArrayList that holds Integers
```

`ArrayList<E>` means "an `ArrayList` of `E`," where you choose `E` when you create it.

The compiler then *guarantees* the list holds only that type.

`ArrayList`: the array that grows

The inconvenience of using arrays is that arrays are **static**

- you must pick the size up front
- remember the word list's "size 600K and hope"?.

`ArrayList` is a **dynamic** array from the `java.util` library that grows automatically as you add items:

```
import java.util.ArrayList;

void main() {
    ArrayList<String> list = new ArrayList<String>();   // empty; grows on demand
    list.add("Boiler");      // [Boiler]
    list.add("Up");          // [Boiler, Up]

    IO.println(list);
}
```

You still pick the element type (`<String>`), just not the size. Compare:

```
String[]            fixed = new String[10];   // exactly 10 slots, forever
ArrayList<String>   grows = new ArrayList<>(); // 0 now, as many as you add
```

Note that repeating the type parameter name on the right is optional.

You can lookup items with `list.get(i)` and find the size with `list.size()`

You can even print an ArrayList nicely with `IO.println(list)` . For arrays, we must use `Arrays.toString(arr)` .

Take Home: Grid Totals

A 2-D array is an array of rows. Summing it needs the nested loop.

Implement both methods for a rectangular grid (every row the same length):

- `int[] rowTotals(int[][] m)` : one total per **row**: `totals[i]` is the sum of row `i`.
- `int[] columnTotals(int[][] m)` : one total per **column**: `totals[j]` is the sum of column `j` across every row.

Sizing hint: how many rows is `m.length`; how many columns is `m[0].length`. The result array's length must match the thing you're totaling.

Press **Test** to run the tests. They use the Boiler Burgers sales grid from lecture (rows should come out `[129, 93, 57]`), a 1×1 grid, a single row, and a single column.